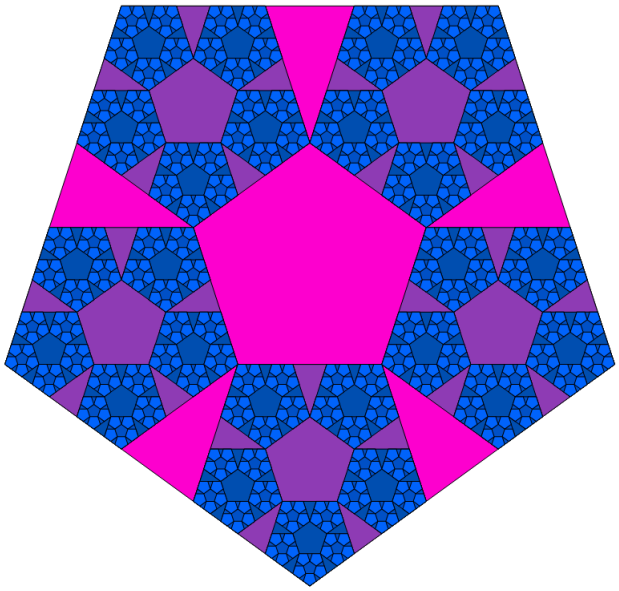
Projet de XML

Systèmes de fonctions itérées



Les *systèmes de fonctions itérées* sont un des procédés utilisés pour générer des images *fractales*, des images présentant certaines similarités à toutes les échelles. L'objectif de ce projet est de représenter de tels systèmes en XML puis d'engendrer, à l'aide de XLST, des rendus de leurs images associées dans le format SVG.

I) Systèmes de fonctions itérées

Soit $\mathcal{P}$ un ensemble quelconque de points du plan.

1. Soit $\mathcal{F} = \{f_1, \ldots, f_n\}$ un ensemble fini de fonctions du plan dans lui-même. On considère la suite d'ensembles de points $(\mathcal{P}_k)_{k\in\mathbb{N}}$ définie inductivement par :

$$\begin{cases} \mathcal{P}_0 & = & \mathcal{P} \\ \mathcal{P}_{k+1} & = & \bigcup_{i=0}^{n} f_i(\mathcal{P}_k) \end{cases}$$

   L'ensemble $\mathcal{P}_k$, noté $\mathcal{F}^{(k)}(\mathcal{P})$, est appelé *l'itération de rang k* de $\mathcal{F}$ sur $\mathcal{P}$. L'association de $\mathcal{P}$ et de $\mathcal{F}$ est appelé un *système (élémentaire) de fonctions itérées.*

2. Soit à présent $\mathfrak{F} = (\mathcal{F}_1, \ldots, \mathcal{F}_m)$ une suite finie d'ensembles finis de fonctions du plan dans lui-même. Pour toute suite d'entiers $(k_1, \ldots, k_m)$, on considère la suite $(\mathcal{Q}_0, \ldots, \mathcal{Q}_m)$ définie par :

$$\begin{cases} \mathcal{Q}_0 & = & \mathcal{P} \\ \mathcal{Q}_j & = & \mathcal{F}_j^{(k_j)}(\mathcal{Q}_{j-1}) \ (1 \leqslant j \leqslant m) \end{cases}$$

   L'ensemble $\mathcal{Q}_m$ est appelé *l'itération de rang $(k_1, \ldots, k_m)$ de $\mathfrak{F}$ sur $\mathcal{P}$*. L'association de $\mathcal{P}$ et de $\mathfrak{F}$ est appelé un *système (composé) de fonctions itérées.*

Autrement dit, une itération de $\mathfrak{F}$ sur $\mathcal{P}$ se construit en calculant d'abord une première itération de $\mathcal{F}_1$ sur $\mathcal{P}$, puis une itération de $\mathcal{F}_2$ sur cette première itération, puis une itération de $\mathcal{F}_3$ sur cette seconde itération, etc.

Dans la suite, on s'intéressera au cas où toutes les fonctions d'un système sont des transformations affines du plan, c'est-à-dire des compositions arbitraires de translations, de rotations, de changements d'échelles, de symétries, etc.

a) Systèmes élémentaires

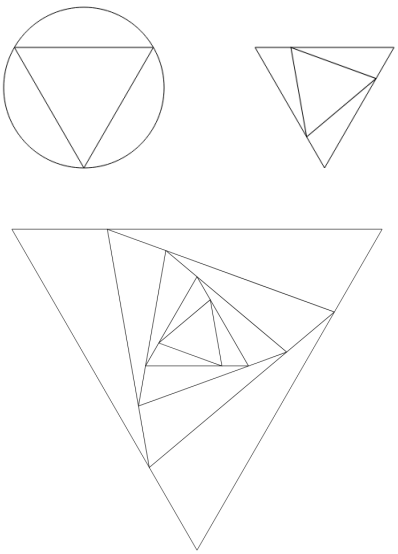
Nous présentons dans cette section des exemples de systèmes élémentaires dont l'ensemble de points initial est un polygone – on pourra bien sûr envisager d'itérer les ensembles de transformations considérés sur des ensembles de points quelconques : déformations de polygones, courbes fermées arbitraires, etc.

Les trois derniers systèmes, dus au mathématicien Wacław Sierpiński (1882–1969), sont bien connus : ils sont des exemples classiques de systèmes de fonctions itérées.

Spirale de base triangulaire. Un exemple de système élémentaire peut être construit ainsi. On part d'un triangle équilatéral inscrit dans le cercle unité [1]. On choisit un angle θ compris entre 0 et 90°, et l'on associe à ce triangle les deux transformations suivantes :

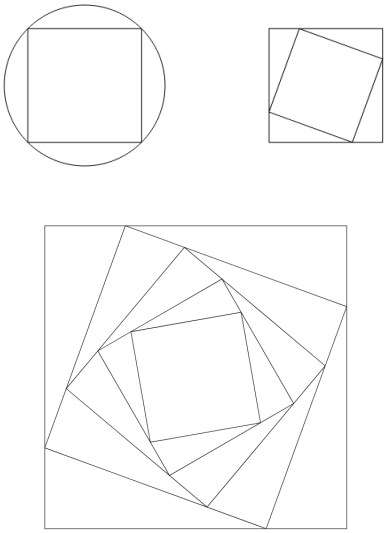
1. L'identité.
2. La composition [2] des transformations suivantes :
   (a) changement d'échelle [3] de ratios (s_x, s_y) avec $s_x = s_y = \frac{1}{\cos(\theta)+\sqrt{3}\times\sin(\theta)}$,
   (b) rotation d'angle θ centrée en l'origine.

Voici un rendu des itérations de rang 0, 1 et 4 de ces transformations sur le triangle pour $\theta = 20°$ soit $s_x = s_y \simeq 0.652703644666$. La première figure indique aussi le placement du triangle dans le cercle unité :

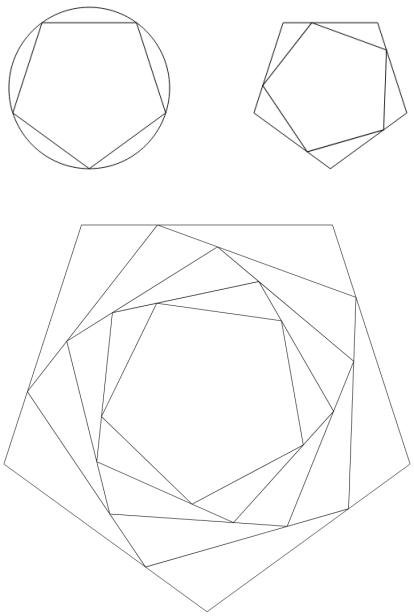


1. Le cercle de rayon 1 centré en l'origine.
2. Les transformations formant une composition sont énumérées dans l'ordre inverse de leur application – dans cet exemple, la rotation est effectuée avant le changement d'échelle. Plus généralement, la composition des transformations $t_1, \ldots, t_n$, habituellement notée $t_1 \circ \ldots \circ t_n$, applique d'abord à un point (x, y) la transformation t_n, puis applique à son résultat la transformation t_{n-1}, puis applique à son résultat la transformation t_{n-2}, etc.
3. Autrement dit, la fonction $(x, y) \mapsto (s_x \times x, s_y \times y)$, avec "s" pour "scale".

Spirale de base carrée. En remplaçant dans le système précédent le triangle par un carré inscrit dans le cercle unité et les valeurs de s_x et s_y par $\frac{1}{\cos(\theta)+\sin(\theta)}$, on obtient pour $\theta = 20°$, soit $s_x = s_y \simeq 0.78020600871$ le rendu suivant :



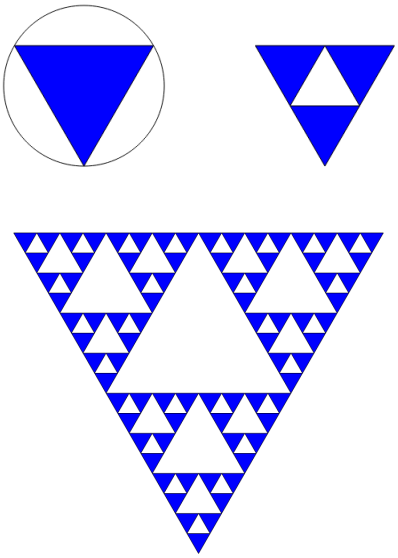
Spirale de base pentagonale. En remplaçant enfin le triangle du premier système par un pentagone régulier et s_x, s_y par $\frac{1}{\cos(\theta)+\tan(36°)\times\sin(\theta)}$, on obtient pour $\theta = 20°$, soit $s_x = s_y \simeq 0.841619922851$:



Triangle de Sierpińsky. Ce système considère un triangle équilatéral inscrit dans le cercle unité, associé à trois transformations définies comme les compositions $t_1 \circ s$, $t_2 \circ s$ et $t_3 \circ s$ où s est le changement d'échelle de ratios (s_x, s_y) avec $sx = sy = \frac{1}{2} = 0.5$, et où :

1. t_1 est la translation de vecteur $(t_x, t_y) = (0, \frac{1}{2}) = (0, 0.5)$
2. t_2 est la translation de vecteur $(t_x, t_y) = (\frac{\sqrt{3}}{4}, -\frac{1}{4}) \simeq (0.433012701892, 0.25)$
3. t_3 est la translation de vecteur $(t_x, t_y) = (-\frac{\sqrt{3}}{4}, -\frac{1}{4})$

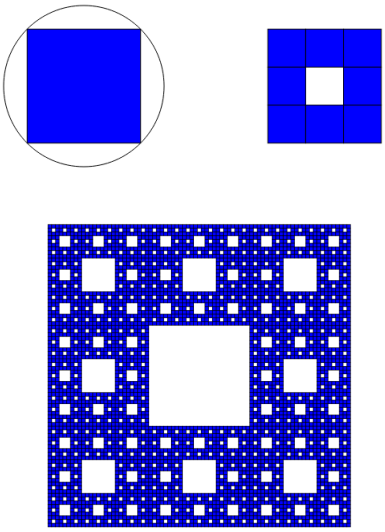
En affichant le bord mais aussi le fond du triangle, les itérations de rang 0, 1 et 4 de ces transformations sur le triangle ont l'aspect suivant :



Tapis de Sierpińsky. On considère un carré inscrit dans le cercle unité et un ensemble de huit transformations de la forme $t^i_j \circ s$ avec $(i,j) \in \{-1,0,1\} \times \{-1,0,1\}$ et $(i,j) \neq (0,0)$, où :

1. s est le changement d'échelle de ratios (s_x, s_y) avec $s_x = s_y = \frac{1}{3} \simeq 0.333333333333$
2. t^i_j est la translation de vecteur $(i \times \frac{\sqrt{2}}{3}, j \times \frac{\sqrt{2}}{3})$ avec $\frac{\sqrt{2}}{3} \simeq 0.471404520791$

Les itérations de rang 0, 1 et 4 ont l'aspect suivant :



Pentagone de Sierpińsky On part d'un pentagone régulier associé à cinq transformations de la forme $t_1 \circ s, \ldots, t_5 \circ s$, où :

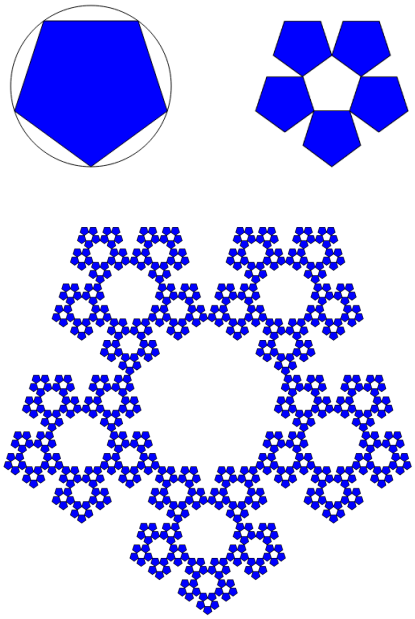
1. s est le changement d'échelle de ratios (σ, σ) avec $\sigma = \frac{3-\sqrt{5}}{2} \simeq 0.38196601125$
2. les $t_1, \ldots, t_5$ sont les translations de vecteurs suivants :
   (a) $(0, \sqrt{\sigma}) \simeq (0, 0.61803398875)$
   (b) $(1-\sigma) \times (\cos(18°), \sin(18°)) \simeq (0.587785252292, 0.190983005625)$

(c) $(1-\sigma)\times(-\cos(18°),\ \sin(18°))$

(d) $(1-\sigma)\times(\cos(54°),\sin(54°))\simeq(0.363271264003,0.5)$

(e) $(1-\sigma)\times(-\cos(54°),\sin(54°))$.

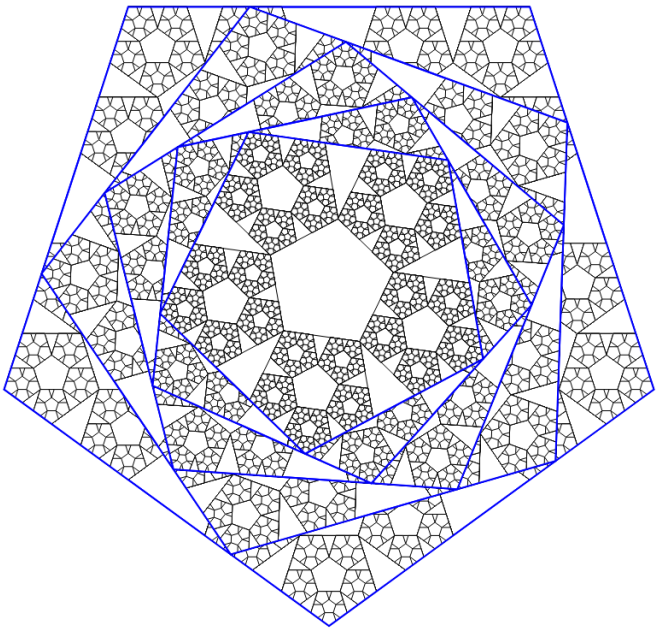
Les itérations de rang $0, 1$ et 4 de ce système sont :



b) Systèmes composés

Soit $\mathcal{P}$ l'ensemble de base du pentagone de Sierpińsky, soit $\mathcal{F}$ l'ensemble formé des cinq transformations de ce système auquel on ajoutera l'identité, et soit $\mathcal{G}$ l'ensemble des transformations de la spirale en 20° dont la base est un pentagone.

Voici à quoi ressemble l'itération de rang $(4,4)$ de $(\mathcal{F},\mathcal{G})$ sur $\mathcal{P}$, c'est-à-dire la quatrième itération de $\mathcal{G}$ sur la quatrième itération de $\mathcal{F}$ sur le pentagone, affiché avec un fond blanc à toutes les échelles. Le contour de chaque copie de la quatrième itération de $\mathcal{F}$ a été surligné en bleu par édition manuelle du fichier `svg` produit :



II) Rendu d'itérations en SVG

Le format SVG se prête naturellement au rendu d'itérations de systèmes, et ce d'une manière relativement concise. Cette possibilité résulte de plusieurs facteurs :

1. On peut regrouper plusieurs éléments SVG (par exemple des éléments de balise `<path/>`, représentant des chemins) en les encadrant par un couple de balises `<g></g>`. Les groupes construits peuvent servir à créer d'autres groupes.
2. Tout élément (un chemin, un groupe, etc) peut subir une transformation affine (rotation, translation, symétrie, composition de transformations, etc) via son attribut `transform`. La valeur de l'attribut `transform` peut être de l'une ou l'autre des formes suivantes :
   - `"translate(`dx`, `dy`)"` : l'élément subit une translation de composantes (dx, dy) ;
   - `"rotate(`a`, `cx`, `cy`)"` : l'élément subit une rotation d'angle a^o et de centre (cx, xy) ;
   - `"scale(`sx`, `sy`)"` : l'élément subit un changement d'échelle de ratios (sx, sy) ;
   - `"`t t'`"` où t et t' sont deux valeurs possibles pour `transform` : l'élément subit la transformation spécifiée par t', puis celle spécifiée par t (noter l'inversion, il s'agit d'une composition) ;
   - `"matrix(`a`, `b`, `c`, `d`, `e`, `f`)"` : l'élément passe de la position (x, y) à la position
   $$(a \times x + c \times y + e, b \times x + d \times y + f).$$

   Remarque. L'épaisseur du tracé d'un chemin (attribut `stroke-width`) est par défaut altérée par les transformations subies par ce chemin (*e.g.* un changement d'échelle de ratios `(2, 2)` multiplie par deux la largeur du tracé), mais on peut imposer à ces transformations d'être neutres pour cette épaisseur en ajoutant au chemin l'attribut suivant :

   ```
   vector-effect="non-scaling-stroke"
   ```

3. Tout élément peut être muni d'un identifiant (via son attribut `id`), et l'on peut se servir plusieurs fois d'un même élément (par exemple dans la construction de groupes) à l'aide de *références* vers cet identifiant (balise `<use/>` et attribut `xlink:href`).

4. Un document SVG peut contenir une section de *définitions*, encadrée par un couple de balises `<defs></defs>`. Les éléments de cette section ne peuvent être affichés de manière directe, mais on peut demander un nombre arbitraire de fois leur inclusion dans le corps principal du document à l'aide de références vers leur identifiant.

III) Représentation de Systèmes de fonctions itérées en XML

a) Travail minimal

Données brutes. Le fichier `csv` fourni avec cet énoncé contient les descriptions, sous forme de chaînes de caractères, des tracés des trois polygones des exemples de cet énoncé, dans le format utilisé par SVG pour décrire les tracés d'éléments de balise `path` par valeur de leur attribut `d` :

```
<path name="triangle" d="M 0 1 L -0.866025404 -0.5 0.866025404 -0.5 z"/>
```

Sans nécessairement avoir à comprendre le détail de la syntaxe de ces chaînes [4], vous trouverez le détail de leur grammaire en suivant les liens de cette page (Section 9.2).

Choix et validation d'un format XML. À partir de ces données brutes, il vous faudra choisir un format XML permettant de représenter des ensembles de points spécifiables par des chemins SVG, mais aussi des ensembles de transformations affines quelconques.

Votre format doit être *validable par schéma*, et il vous est demandé d'écrire un fichier `xsd` permettant cette validation. Votre schéma devra être le plus contraint possible – par exemple, si certaines données sont représentées par des chaînes de caractères, la grammaire de ces chaînes devra être spécifiée [5] de manière à interdire toute chaîne mal formée.

Il vous faudra représenter dans le format choisi au moins les ensembles de transformations et les polygones présentés dans cet énoncé.

Génération de rendus avec XSLT. La dernière étape sera d'écrire un [6] script XLST permettant, à partir des données XML, de construire des rendus d'itérations au format SVG des systèmes élémentaires ou composés engendrés par ces transformations et ces chemins, en particulier les rendus de cet énoncé. L'utilisateur devra pouvoir spécifier par des options sur la ligne de commande :

- le chemin servant de base à ce rendu,
- l'ensemble de transformations à utiliser, ou les ensembles de transformations à composer,
- le rang de l'itération de ce rendu,
- toute autre information utile : dimensions du document produit et échelle du rendu, remplissage, ajout de l'identité à l'un des ensembles de transformations, etc.

b) Extensions

De nombreuses extensions sont envisageables.

- Recherche de nouveaux tracés et de nouveaux ensembles de transformations, en particulier si leurs rendus sont d'une qualité esthétique intéressante.

4. Si vous souhaitez considérer de nouvelles figures, un éditeur SVG tel que `inkscape` peut les générer pour vous. Les valeurs d'attributs `d` associées peuvent être facilement extraites des fichiers SVG produits.
5. Ceci peut être moins simple que de représenter la même information par une imbrication d'éléments, en spécifiant des contraintes sur cette imbrication.
6. Ou plusieurs, si le traitement est en plusieurs phases.

- On peut ajouter des couleurs à un rendu (*c.f.* la toute première figure de l'énoncé pour un essai parmi d'autres : dégradés sur les composantes rouge et bleu).
- Le rendu dans un navigateur peut être animé (élément `animateTransform`), pourvu que les transformations soient spécifiées sous forme non matricielle.
- Les arguments des transformations peuvent être des approximations de valeurs d'expressions arithmétiques (*e.g.* 0.38196601125 comme approximation de $(3 - \sqrt{5})/2$), éventuellement paramétriques (*e.g.* $1/(\cos(\theta) + \sin(\theta))$ pour une valeur de θ à spécifier).

  Rien n'interdit de représenter de telles expressions en XML (elles ont une représentation arborescente) ni d'ajouter à un fichier XSLT un mécanisme permettant leur évaluation, éventuellement en fonction de paramètres dont les valeurs seront spécifiées sur la ligne de commande.

IV) Modalités

a) Binômes

Le projet doit être impérativement réalisé en binôme. Toutefois, en cas d'impossiblité majeure de satisfaire cette contrainte (nombre impair d'étudiants, conditions d'études particulières, etc.), nous vous invitons à en discuter le plus tôt possible avec votre enseignant de cours magistral - aucun travail non réalisé en binôme ne sera accepté sans son accord explicite. La répartition des tâches au sein d'un binôme doit être raisonnablement équilibrée. En cas de déséquilibre avéré, les notes finales pourront être individualisées.

b) Usage de Gitlab

Ce travail devra être réalisé via un dépôt git hébergé par le serveur `GitLab` de l'UFR d'informatique : https://moule.informatique.univ-paris-diderot.fr. Aucun autre code ne sera pris en compte, même hébergé par un autre serveur.

Votre dépôt doit être rendu **privé** dès sa création, avec accès uniquement aux membres du groupe et aux enseignants de ce cours. Il va de soi que votre travail doit être strictement personnel : aucune communication de code ou d'idées entre les groupes, aucune "aide" externe ou entre groupes [7].

c) Forme du rendu

La date limite du dernier commit sur la branche master de votre dépôt ainsi que celle des soutenances seront précisées ultérieurement. Au moment de la soutenance, cette branche devra contenir :

1. Les fichiers `xml`, `xsd` et `xsl` du projet (laissez le `csv` et les `svg` produits en `gitignore`).
2. Un fichier texte `README` contenant vos noms et numéros d'étudiants, et précisant la manière d'utiliser vos fichiers.
3. Tout autre fichier utile (*e.g.* script ou `Makefile`).

V) Références

– https://en.wikipedia.org/wiki/Iterated_function_system
La notion de système de fonctions itérées est à la fois plus générale que celle présentée ici – elle ne se limite pas aux fonctions du plan dans lui-même – et plus restrictive – on impose aux fonctions d'être *contractantes*, intuitivement de raccourcir les distances, ce qui est le cas de toutes les fonctions des exemples ci-dessus mais n'est pas imposé par l'énoncé.

– https://larryriddle.agnesscott.org/ifs/ifs.htm
Vous trouverez sur cette page des exemples supplémentaires de fractales générées par systèmes de fonctions itérées (voir également la galerie du site).

– https://www.w3.org/TR/SVG2
https://developer.mozilla.org/en-US/docs/Web/SVG
Le premier lien est le sommet de la norme officielle de SVG, le second mène à des tutoriaux et de très nombreux exemples d'usages de ce format.

7. Que ce soit sa réalisation ou sa soutenance, le projet est un examen comme un autre. Le plagiat en projet constitue une fraude aux examens passible de lourdes sanctions discplinaires – ce cas s'est produit hélas plus d'une fois dans cette UFR.